

FruGo

Your private deep-dive — every feature explained with WHY, not just WHAT.

OTP Auth

Session Cart

Order Status Tabs

Blade Templates

MailHog

Vercel Deploy

CHAPTER 01

OTP Authentication — Step by Step Through the Code

OTP stands for One-Time Password. Instead of creating a password, the customer just enters their email. A 6-digit code is emailed to them, they enter it, and they're logged in. Let's trace the exact code path.

4

8

2

9

1

3

1 Customer visits /login-otp and enters email

OtpLoginController@request handles GET (show form) and POST (submit email)

2 Controller generates 6-digit code

`random_int(100000, 999999)` — cryptographically random, not predictable

3 Code saved to otp_tokens table

Stored with the email, hashed code, and expires_at = now + 10 minutes

4 Email dispatched

`Mail::to($email)->send(new OtpMail($code))` — MailHog catches it locally

5 Customer enters 6 digits

/otp-verify form — 6 input boxes, auto-focus next on each keypress

6 Controller verifies code

Looks up token by email, checks not expired, checks code matches

7

Token deleted, user logged in

`$token->delete()` then `Auth::login($user)`. First-time: `User::firstOrCreate()` auto-creates account.

app/Http/Controllers/Auth/OtpLoginController.php – sendOtp()

PHP

```
public function sendOtp(Request $request): RedirectResponse
{
    $request->validate(['email' => ['required', 'email']]);

    // Generate a 6-digit number – PHP's cryptographically secure random
    $code = random_int(100000, 999999);

    // Delete any old tokens for this email before creating a new one
    // Prevents the same person from having 3 valid codes at once
    OtpToken::where('email', $request->email)->delete();

    OtpToken::create([
        'email'      => $request->email,
        'token'      => bcrypt($code),    // stored as hash, not plaintext
        'expires_at' => now()->addMinutes(10),
    ]);

    Mail::to($request->email)->send(new OtpMail($code));

    return redirect()->route('otp.verify.form')
        ->with('email', $request->email);
}
```

WHY HASH THE OTP TOKEN IN THE DATABASE?

If someone got read access to your database (SQL injection, compromised backup), they'd see all pending OTPs. If stored as plain numbers, they could log in as anyone. By storing `bcrypt($code)`, the stored value is useless without the original number. The controller calls `Hash::check($submitted, $stored)` to verify without ever storing plaintext.

The otp_tokens Table — How It's Designed

The OTP system needs its own database table. Let's look at its migration and understand each column.

database/migrations/create_otp_tokens_table.php

PHP

```
Schema::create('otp_tokens', function (Blueprint $table) {
    $table->id();                // auto-increment primary key
    $table->string('email');      // whose email this token is for
    $table->string('token');      // bcrypt'd OTP code
    $table->timestamp('expires_at'); // now + 10 minutes
    $table->timestamps();        // created_at, updated_at

    // Index on email so the lookup query is fast
    $table->index('email');
});
```

OtpLoginController - verifyOtp() - checking the token

PHP

```
public function verifyOtp(Request $request): RedirectResponse
{
    // The 6 input boxes are named digit_1 to digit_6
    // Merge them into a single 'code' string before validation
    $code = implode('', [
        $request->digit_1, $request->digit_2, $request->digit_3,
        $request->digit_4, $request->digit_5, $request->digit_6,
    ]);

    // Find the most recent unexpired token for this email
    $otpToken = OtpToken::where('email', $request->email)
        ->where('expires_at', '>', now())
        ->latest()
        ->first();

    if (!$otpToken || !Hash::check($code, $otpToken->token)) {
        return back()->withErrors(['code' => 'Invalid or expired code']);
    }
}
```

```

// Delete immediately – cannot be reused
$otpToken->delete();

// firstOrCreate: if user exists → log them in; if not → create + log in
$user = User::firstOrCreate(
    ['email' => $request->email],
    ['name' => explode('@', $request->email)[0], 'role' => 'customer']
);

Auth::login($user);
$request->session()->regenerate(); // prevents session fixation attack

return redirect()->route('shop.index');
}

```

EXAMPLE – WHAT FIRSTORCREATE DOES

```
User::firstOrCreate(['email' => 'alice@example.com'], ['name' => 'alice', 'role' => 'customer'])
```

First argument: search criteria. Second argument: values to set if creating.

If alice@example.com already exists → returns that user.

If alice@example.com is new → creates a user with name='alice', role='customer', returns that user.

Either way, the result is a User object ready to log in.

CHAPTER 03

Session Cart — No Database Needed

The cart stores what products the customer wants to buy. We use PHP's **session** instead of the database for this. A session is a small chunk of data that Laravel stores between requests, tied to the customer's browser.

ANALOGY

A session is like a **sticky note on the waiter's notepad** — it stays attached to your table for the duration of your visit. When you add an item to the cart, the waiter writes it down. When you leave

(log out / session expires), the note is thrown away. No need to file it in a cabinet (database) because cart contents don't need to persist forever.

app/Http/Controllers/Shop/CartController.php – addToCart()

PHP

```
public function add(Request $request, int $productId): RedirectResponse
{
    // Get current cart – default to empty array if nothing yet
    $cart = $request->session()->get('cart', []);

    // Cart is an array: [ product_id => quantity ]
    // e.g. [ 5 => 2, 12 => 1 ] = 2 of product 5, 1 of product 12
    if (isset($cart[$productId])) {
        $cart[$productId]++; // already in cart? increase quantity
    } else {
        $cart[$productId] = 1; // new product? start at 1
    }

    // Write the updated cart back to the session
    $request->session()->put('cart', $cart);

    return back()->with('success', 'Added to cart');
}
```

CartController@viewCart – loading product details from session IDs

PHP

```
public function index(Request $request): View
{
    // Session cart: [ 5 => 2, 12 => 1 ]
    $cartData = $request->session()->get('cart', []);

    // One query to get ALL products at once (not one per product)
    // array_keys() extracts [5, 12] – the product IDs
    $products = Product::whereIn('id', array_keys($cartData))->get();

    // Calculate total: sum(price × quantity) for each product
    $total = $products->sum(fn($p) => $p->price * $cartData[$p->id]);

    return view('shop.cart', compact('products', 'cartData', 'total'));
}
```

CONCEPT — WHEREIN

`Product::whereIn('id', [5, 12])` becomes SQL: `SELECT * FROM products WHERE id IN (5, 12)`. This gets all products in one query instead of one query per product. Without `whereIn`, a cart with 5 items would run 5 separate queries — inefficient.

Cart count in navbar — zero queries:

resources/views/layouts/shop.blade.php — navbar cart count

Blade

```
{{-- array_sum() adds up all quantities in the cart session array --}}
{{-- e.g. [5=>2, 12=>1] → sum = 3 items --}}
{{-- No database query – reads from session in memory --}}
Cart ({{ array_sum(session('cart', [])) }})
```

CHAPTER 04

Cart to Order — How Checkout Works

When the customer places an order, the session cart gets converted to permanent database records (orders + order_items), and the session is cleared.

OrderController@store — placing an order

PHP

```
public function store(Request $request): RedirectResponse
{
    $cartData = $request->session()->get('cart', []);

    // Wrap everything in a DB transaction
    // If any step fails, the whole thing rolls back – no partial orders
    DB::transaction(function() use ($request, $cartData) {
        $products = Product::whereIn('id', array_keys($cartData))->get();
        $total     = $products->sum(fn($p) => $p->price * $cartData[$p->id]);

        // Create the parent order record
        $order = Order::create([
            'user_id'      => auth()->id(),
```

```

        'total_amount'    => $total,
        'delivery_address' => $request->delivery_address,
        'status'          => 'pending',
    ]);

    // Create one order_item row per product in the cart
    foreach ($products as $product) {
        $order->items()->create([
            'product_id' => $product->id,
            'quantity'   => $cartData[$product->id],
            'price'       => $product->price,
        ]);
    }
});

// Clear the cart session after successful order
$request->session()->forget('cart');

return redirect()->route('shop.orders.index')
    ->with('success', 'Order placed!');
}

```

CONCEPT — DB TRANSACTION

A transaction means "do all of this, or none of it." If creating the order succeeds but then the server crashes before creating order_items, the whole order rolls back. Without a transaction, you'd have an order with no items — a partial, broken record. Always use transactions when writing to multiple tables that must stay in sync.

CHAPTER 05

Order Status Tabs — How the Filtering Works

The order history page has 5 tabs: All, Pending, In Process, Completed, Cancelled. Each tab shows only orders in that status. The active tab is stored in the URL as `?status=pending` so you can bookmark or share a filtered view.

```

public function index(Request $request): View
{
    // Base query: only this customer's orders
    $base = Order::where('user_id', auth()->id());

    // Clone the base query for each tab count — efficient!
    // clone() copies the query builder without running it
    $counts = [
        'all'      => (clone $base)->count(),
        'pending'  => (clone $base)->where('status','pending')->count(),
        'processing' => (clone $base)->where('status','processing')->count(),
        'completed' => (clone $base)->where('status','completed')->count(),
        'cancelled' => (clone $base)->where('status','cancelled')->count(),
    ];

    // Filter the main query based on ?status= parameter
    $status = $request->input('status', 'all');
    if ($status !== 'all') {
        $base->where('status', $status);
    }

    // appends() keeps ?status= in pagination links (?status=pending&page=2)
    $orders = $base->latest()->paginate(10)->appends($request->query());

    return view('shop.orders', compact('orders', 'counts', 'status'));
}

```

WHY CLONE THE BASE QUERY FOR COUNTS?

If you built `Order::where('user_id', auth()->id())` 5 times, you'd repeat yourself. By cloning the base query, you get all 5 counts with the same user filter, each adding one more `where`. Also, `->count()` only runs `SELECT COUNT(*)` — no row data fetched. Very fast even with thousands of orders.

```

{{-- Loop through each status, highlight the active one --}}
@foreach(['all','pending','processing','completed','cancelled'] as $tab)
    <a href="{{ route('shop.orders.index', ['status' => $tab]) }}"

```



```

        class="tab {{ $status === $tab ? 'active' : '' }}">
        {{ ucfirst($tab) }} ({{ $counts[$tab] }})
    </a>
@endforeach

```

CHAPTER 06

Blade Shop Templates — {{ }} vs {!! !!}

Blade is Laravel's template engine. It converts `.blade.php` files into plain HTML. The two most important things to understand: how to safely output data, and how layouts work.

Blade output syntax – safe vs unsafe

Blade

```

{{-- SAFE: {{ }} HTML-encodes the value --}}
{{-- <script> becomes &lt;script> – shown as text, not executed --}}
<h2>{{ $product->title }}</h2>

{{-- UNSAFE: {!! !!} outputs raw HTML -- use only on pre-sanitised content --}}
{{-- We use this for product description ONLY because we strip_tags'd it already --}}
<div class="description">{!! $product->description !!}</div>

{{-- @if, @foreach, @forelse are Blade's control flow --}}
@forelse($products as $product)
    <div class="product-card">...</div>
@empty
    <p>No products found.</p>
@endforeach

{{-- @csrf generates a hidden input with the CSRF token --}}
<form method="POST" action="{{ route('cart.add', $product) }}">
    @csrf
    <button>Add to Cart</button>
</form>

```

FruGo has a shared layout file `resources/views/layouts/shop.blade.php` — it has the navbar, footer, CSS links. Individual pages (shop/index, cart, orders) use `@extends('layouts.shop')` to inherit that layout. Then `@section('content') ... @endsection` fills in the main body area. No repeating the navbar on every page.

CHAPTER 07

Sticky Navbar — The CSS Trick

FruGo has two header elements stacked. The green topbar scrolls away with the page. The white FruGo navbar stays pinned to the top of the viewport as you scroll down.

Green topbar (scrolls away)

Normal page flow — `position: static`. When you scroll past it, it disappears off the top of the screen.

Contains: phone number, email, social links.

White FruGo navbar (stays)

`position: sticky; top: 0` — pins to the top of the viewport once the topbar has scrolled away.

Contains: logo, search, cart count.

The CSS that makes it stick

CSS

```
/* position: sticky means: normal flow until the element would scroll off,
   then pin it at 'top: 0' — the top of the viewport */
.site-navbar {
  position: sticky;
  top: 0;
  z-index: 1000;
  background: white;
}

/* JS adds a deeper shadow as you scroll — gives depth feedback */
/* The $(window).scroll handler adds/removes a .scrolled class */
.site-navbar.scrolled {
  box-shadow: 0 2px 20px rgba(0, 0, 0, 0.15);
}
```

WHY STICKY INSTEAD OF FIXED?

`position: fixed` removes the element from page flow entirely — it would overlap your content from pixel one. `position: sticky` keeps it in page flow until it "hits" the top, then pins it. The green topbar scrolls normally; once it's gone, the navbar sticks. Much cleaner than fixed + margin-top hacks.

CHAPTER 08

Custom Bootstrap 5 Pagination

Laravel's default Bootstrap 5 pagination uses Bootstrap Icons (`bi-chevron-left`) which didn't render correctly in FruGo's font setup — they showed as large boxes. The fix: publish Laravel's pagination view and replace the icons.

resources/views/vendor/pagination/bootstrap-5.blade.php

Blade

```
{{-- Replaced Bootstrap Icon spans with FontAwesome --}}
{{-- Also restyled as circular pill buttons --}}
@if ($paginator->onFirstPage())
    <li class="page-item disabled">
        <span class="page-link">
            <i class="fas fa-chevron-left" style="font-size:.75rem"></i>
        </span>
    </li>
@else
    <li class="page-item">
        <a class="page-link" href="{{ $paginator->previousPageUrl() }}">
            <i class="fas fa-chevron-left" style="font-size:.75rem"></i>
        </a>
    </li>
@endif
```

CONCEPT — PUBLISHING VENDOR VIEWS

Laravel's core files live in `vendor/` — you can't edit them because they'd be overwritten on the next `composer update`. Publishing copies the view to your `resources/views/vendor/` folder, where Laravel

will always use *your copy* over the package's copy. Command: `php artisan vendor:publish --tag=laravel-pagination`

CHAPTER 09

Owl Carousel Nav Arrow Fix

The FruGo home page uses Owl Carousel for a product/testimonials slider. By default, Owl Carousel renders large left/right navigation arrows. These overlapped the product listings. Two fixes were applied — in JS and CSS.

public/js/main.js – disable nav in carousel init

JavaScript

```
// nav: false = don't generate the arrow buttons at all
$('.owl-carousel.testimonials').owlCarousel({
  nav: false,
  dots: true,
  loop: true,
  items: 1
});

$('.owl-carousel.vegetables').owlCarousel({
  nav: false,
  responsive: { 0:{items:2}, 768:{items:4} }
});
```

CSS safety net – kills any carousel nav that slips through

CSS

```
/* Belt-and-suspenders: if any other carousel opts into nav,
   this CSS rule hides the buttons so they never overlap content */
.owl-carousel .owl-nav {
  display: none !important;
}
```

WHY BOTH JS AND CSS?

`nav: false` is the right fix — no buttons generated. The CSS is a safety net for the future. If someone adds another carousel and forgets to set `nav: false`, the CSS catches it. Defence in depth: fix the problem, then add a catch-all.

CHAPTER 10

MailHog — Catch Emails in Development

FruGo's OTP feature sends emails. In development, you don't want real emails going out. MailHog is a fake email server running in Docker that catches every outgoing email and shows it in a web UI.

docker-compose.yml — MailHog service

YAML

```
mailhog:
  image: mailhog/mailhog:latest
  ports:
    - "8025:8025" # web UI — open in browser
    - "1025:1025" # SMTP port — Laravel sends here
  networks:
    - frugo_network
```

1 Customer requests OTP

Laravel calls `Mail::to($email)->send(new OtpMail($code))`

2 Laravel sends to mailhog:1025

`.env` has `MAIL_HOST=mailhog`, `MAIL_PORT=1025` — the Docker service name resolves automatically

3 MailHog catches it

Never goes to the real internet. Stored in MailHog's in-memory inbox.

4 You view it at localhost:8025

Open the URL, click the email, see the full HTML rendering with the OTP code clearly shown.

EXAMPLE — THE OTP EMAIL MAILABLE CLASS

Laravel Mailables are PHP classes that represent an email. `app/Mail/OtpMail.php` takes the OTP code in its constructor, and its `content()` method points to the `resources/views/emails/otp.blade.php` template. The template shows the code with a 10-minute expiry warning. When `Mail::to()-`

`>send(new OtpMail($code))` is called, Laravel renders the Blade view, wraps it in an email envelope, and sends it.

CHAPTER 11

Vercel Deployment — PHP on a Serverless Platform

Vercel is a platform designed for JavaScript (Next.js). Running PHP on it requires a community runtime and some tricks to work around its read-only filesystem. Here's how it was made to work.

CONCEPT — SERVERLESS VS TRADITIONAL SERVER

A traditional server (like your Docker setup) is always running. One PHP-FPM process handles requests. It can write files to disk because it has a real filesystem. Vercel is **serverless** — each request starts a fresh, short-lived container. The filesystem is read-only except for `/tmp`. The container is destroyed after the request. Laravel needs writable storage — so we route it to `/tmp`.

api/index.php — entry point (creates /tmp dirs before Laravel boots)

PHP

```
if (!empty($_ENV['VERCEL'])) {
    // Vercel filesystem is read-only except /tmp
    // Create these directories every request (they vanish between requests)
    $dirs = [
        '/tmp/storage/framework/sessions',
        '/tmp/storage/framework/views',
        '/tmp/storage/framework/cache/data',
        '/tmp/storage/logs',
    ];
    foreach ($dirs as $dir) {
        if (!is_dir($dir)) mkdir($dir, 0755, true);
    }
}
```

bootstrap/app.php — redirect storage path to /tmp on Vercel

PHP

```

$app = Application::configure(basePath: dirname(__DIR__))...
    ->create();

// Tell Laravel to use /tmp/storage instead of /var/www/storage
// This makes all framework writes (views cache, session, logs) go to /tmp
if (!empty($_ENV['VERCEL'])) {
    $app->useStoragePath('/tmp/storage');
}

return $app;

```

WHY SESSION_DRIVER=COOKIE ON VERCEL?

Normally Laravel stores sessions as files in `storage/framework/sessions/`. On Vercel, each request runs in a different container — container A's session files don't exist in container B.

`SESSION_DRIVER=cookie` stores the session data in an encrypted cookie in the browser instead. It travels with every request regardless of which container handles it. Same reason `CACHE_STORE=array` is used — array cache only lives for the duration of one request, which is fine when there's no shared cache between containers.

CHAPTER 12

Security in FruGo — Everything That Protects Customers

Security isn't one thing — it's a series of layers, each blocking a different type of attack. Here's every protection in FruGo and why each one matters.

ATTACK	HOW FRUGO STOPS IT	WHERE IN CODE
OTP Replay	Token deleted immediately on successful verify. Expires in 10 minutes. Old tokens deleted before issuing a new one.	OtpLoginController@verifyOtp
SQL Injection	All queries use Eloquent's parameterised builder. User input in ?search= is never concatenated into SQL.	ProductRepository, CartController

ATTACK	HOW FRUGO STOPS IT	WHERE IN CODE
CSRF	@csrf on every POST form. VerifyCsrfToken middleware rejects requests without a valid token.	All Blade forms
Session Fixation	session()->regenerate() called on login. Regenerates session ID so a pre-login session ID can't be reused.	OtpLoginController@verifyOtp
Privilege Escalation	Customers cannot reach /products or /admin. CheckRole middleware + auth guard on all staff routes.	routes/web.php + CheckRole.php
Sensitive Env Vars	.env gitignored. Only .env.example committed. Vercel env vars set via dashboard, never in code.	.gitignore

CONCEPT – CSRF (CROSS-SITE REQUEST FORGERY)

Without CSRF protection, a malicious website could make your browser silently send a POST request to FruGo: "add 100kg of mangoes to cart and checkout." Your browser sends your session cookie automatically, so FruGo would think *you* sent that request. The `@csrf` token is a secret value only your FruGo session knows. The malicious site can't know it, so its fake request fails.